

SESSION 1. Introduction to Database &SQL , Creation of Tables

SESSION 2. Datatypes & Sub Languages -1

SESSION 3. Sub Languages -2

SESSION 4. Sub Languages -3 & Integrity Constraints -1

SESSION 5. Integrity Constraints -2

SESSION 6. Limitations of Excel & Intro to Real Time Database

SESSION 7. Operators & Clauses -1

SESSION 8. Operators & Clauses -2

SESSION 9. Installation of MySQL & Integrity Constraints -1

SESSION 10. Integrity Constraints -2, Operators & Clauses -1

SESSION 11. Operators & Clauses -2

SESSION 12. Like Operator & Functions -1

SESSION 13. Functions -2

SESSION 14. Functions -3 & Regular Expression

SESSION 15. Joins -1

SESSION 16. Joins -2 - Examples on Joins

SESSION 17. Joins -3 - Examples on Joins

SESSION 18. GROUP BY & Having Clause

SESSION 19. Sub Queries – 1

SESSION 20. Sub Queries - 2 & Joins Assignment Solutions

SESSION 21. Views & ER - Diagram -1

SESSION 22. ER Diagram -2 & Indexes

SESSION 23. Temporary Tables & CTE -1

SESSION 24. CTE – 2

SESSION 25. Stored Procedures & Window Functions -1

SESSION 26. Window Functions -2

SESSION 27. Denormalization, Normalizations & Normal Forms

SESSION 28. Interview Questions

SESSION 29. MCQ – 2

SESSION 30. MCQ-1

SESSION 1. Introduction to Database&SQL , Creation of Tables

Core Theme 1: Importance of Data & SQL

- **Data Explosion:** Growth driven by online shopping, digital payments, remote work, social media, OTT platforms.
- **Corporate Data Usage:** Companies collect and analyze user data (name, age, gender, location) to improve products/services.
 - Example: Netflix analyzes viewing habits by age and gender to inform content strategy.
- **Data Science & Analytics:** Used across industries (banking, e-commerce, medical) to enhance decisions and profitability.
- **SQL as a Key Skill:**
 - Essential for communicating with databases.
 - Backbone of IT industry; enables querying, analysis, and data-driven decisions.

Core Theme 2: Fundamental Database Concepts

- **Analogy:** SQL is like the language used to retrieve information from a database, similar to asking a peon to fetch a register.
- **Key Terms:**
 - **Database:** Organized storage of large volumes of data.
 - **SQL:** Structured Query Language, universal for relational databases.
 - **Table:** Stores data in rows and columns.
 - **RDBMS:** Relational Database Management System; organizes tables with relationships.
 - **Relational Tables:** Linked via common columns, e.g., employee_ID or manager_ID.
 - **SQL vs NoSQL:** SQL uses relational tables; NoSQL (e.g., MongoDB) is non-relational.
 - **SQL vs MySQL:** SQL is the language; MySQL is an open-source database product. Syntax is largely transferable across platforms.

Core Theme 3: MySQL Practical Application

- **MySQL Workbench UI:**
 - **Schemas Panel:** Lists databases
 - **Information Section:** Displays table metadata

- **Query Editor:** Write and execute SQL queries
- **Output Section:** Shows query results and status
- **Toolbar Icons:** Save, open, execute queries
- **Basic SQL Commands:**
 - **Case-Insensitive Commands:** Can use uppercase or lowercase
 - **Comments:**
 - Single-line: -- or #
 - Multi-line: /* ... */
 - **Create Database:** CREATE DATABASE SQL_8;
 - **Use Database:** USE SQL_8;

Detailed Course Syllabus

Part 1: Foundational Concepts

- Introduction to Database & SQL
- Limitations of Excel for data storage
- Creating Databases & Tables
- MySQL Installation
- Data Types in SQL
- SQL Sublanguages: DDL, DML, DCL, TCL, DQL
- Integrity Constraints: Check, Default, Primary/Foreign/Unique keys

Part 2: Core & Advanced Topics (Practical "Movies Database")

- **Operators & Clauses:** WHERE, ORDER BY, DISTINCT, LIMIT, AND, OR, NOT, BETWEEN, IN, LIKE (wildcards)
- **Functions:** Numerical, Character, Date, Aggregate
- **Joins (Backbone of SQL):** Inner, Left, Right, Outer, Self, Cross, Natural
- **Regular Expressions:** Advanced pattern matching with anchors
- **Advanced Clauses:** GROUP BY, HAVING, aggregates
- **Subqueries:** Correlated and Non-correlated (Single & Multi-row)
- **Normalization & Denormalization:** Conversion to 1NF, 2NF, 3NF

- **ER Diagrams**
- **Views, Indexes, Temporary Tables:** Practical use cases like removing duplicates
- **CTEs:** Advanced querying
- **Stored Procedures:** Reusable blocks, IN/OUT parameters
- **Window Functions:** RANK(), DENSE_RANK(), ROW_NUMBER(), LEAD(), LAG()
- **Interview Focus:** Important questions discussed at course end

Pedagogical Approach & Logistics

- **Hands-On Focus:** Emphasis on practical exercises over theory
- **Course Notes:** Delivered as actual SQL files with comments, syntax, and executed examples
- **Material Access:** Daily videos and SQL files provided upon registration
- **Batch Start:** Formal teaching begins April 25th

Key Clarifications

- **Scope:** Course covers SQL only, not PL/SQL (cursors, exception handling, advanced optimization)
- **Skill Transferability:** MySQL syntax transferable to other RDBMS like SQL Server or Oracle (99% same)

Session 2. Datatypes & Sub Languages -1

1. Importance of SQL and Data Types

- SQL databases enforce **data integrity** and prevent irrelevant or incorrect entries, unlike spreadsheets (Excel).
- **Excel Limitations:**
 - Maximum ~1 million rows (limited scalability).
 - No strict data type enforcement → can enter inconsistent data (e.g., text in a numeric column).
- **SQL Solution:**
 - Each column must have a defined data type.
 - Inserting values that do not match the column's data type triggers an error, ensuring data consistency.

2. Core SQL Data Types

Numerical

- **INT:** Stores whole numbers (e.g., Employee ID, quantities).
- **FLOAT:** Stores decimals and integers (e.g., Salary, Price).

Character/String

- **CHAR(size):** Fixed-length; pads unused space with blanks → memory wastage; max length 255.
- **VARCHAR(size):** Variable-length; memory-efficient; max length 24,534; preferred for names, addresses.

CHAR vs VARCHAR

- **CHAR:** Uses full defined memory regardless of actual data length → inefficient for variable data.
- **VARCHAR:** Uses memory proportional to data length → efficient, avoids waste.
- Real-world use: CHAR for fixed-length data (e.g., Gender), VARCHAR for variable-length data (e.g., Names).

Date/Time

- **DATE:** Stores only date, format 'YYYY-MM-DD'.
- **DATETIME:** Stores date and time, format 'YYYY-MM-DD HH:MM:SS'.

Boolean

- **BOOLEAN:** Stores TRUE/FALSE or 1/0; used for binary states like status.

3. Fundamental Table Operations

- **Activate Database:** USE database_name;

- **Create Table:** Defines structure, columns, and data types.
- **Insert Data:**
 - Values must match column order and data type.
 - Character and Date values need quotes; numerical values do not.
 - Multiple rows can be inserted in one command.
- **Display Data (SELECT):**
 - `SELECT * FROM table_name;` → all columns and rows.
 - `SELECT column1, column2 FROM table_name;` → specific columns.

4. SQL Command Sublanguages

Acronym	Full Name	Purpose	Example Commands
DDL	Data Definition Language	Defines/manages table structure	CREATE, ALTER, DROP, RENAME, TRUNCATE
DML	Data Manipulation Language	Manages table data	INSERT, UPDATE, DELETE
DQL/DRL	Data Query/Retrieval Language	Retrieves/query data	SELECT
DCL	Data Control Language	Manages user permissions	GRANT, REVOKE
TCL	Transaction Control Language	Ensures data integrity in transactions	COMMIT, ROLLBACK, SAVEPOINT

5. Data Definition Language (DDL) In-Depth

- **DROP:** Permanently deletes a table or database.
 - Example: `DROP TABLE employees1;` → irreversible removal.
- **ALTER:** Modifies existing table structures.
 - Add column: `ALTER TABLE employees ADD bonus FLOAT;`
 - Remove column: `ALTER TABLE employees DROP COLUMN bonus;`
 - Modify column type/size: `ALTER TABLE employees MODIFY COLUMN employee_name VARCHAR(30);`
 - Rename column: `ALTER TABLE employees RENAME COLUMN doj TO date_of_join;`

6. SQL Syntax and Execution Notes

- **Semicolon (;):**
 - Optional for single statements.
 - Mandatory for multiple statements to separate queries.
- **Error Handling:**
 - Violating a data type constraint triggers clear error messages.
 - Example: Inserting a name longer than CHAR(10) → Data too long for column 'employee_name' at row 1.

Session 3. Sub Languages -2

SQL Sublanguages: Key Insights

1. Overview of SQL Sublanguages

- SQL commands are categorized into **five sublanguages** based on function:
 - DDL (Data Definition Language):** Defines/manages database structure (tables, columns).
 - DML (Data Manipulation Language):** Manipulates data within tables.
 - DCL (Data Control Language):** Manages user permissions and access control.
 - TCL (Transaction Control Language):** Controls transactions in the database.
 - DQL (Data Query Language):** Queries and retrieves data.

2. DDL (Data Definition Language) Commands

- Purpose:** Create and modify the structure of database objects.

ALTER

- Add Column:** ALTER TABLE table_name ADD column_name datatype;
- Remove Column:** ALTER TABLE table_name DROP COLUMN column_name;
- Modify Column Type:**
 - MySQL: ALTER TABLE table_name MODIFY COLUMN column_namenew_data_type;
 - SQL Server: ALTER TABLE table_name ALTER COLUMN column_namenew_data_type;
- Rename Column:** ALTER TABLE table_name RENAME COLUMN old_name TO new_name;
- Add Primary Key:** Adds a primary key constraint to an existing table.

TRUNCATE

- Removes **all rows** from a table but preserves table structure.
- Syntax: TRUNCATE TABLE table_name;
- Use Case: Quickly replace table data without dropping the table.

RENAME

- Changes the name of a table.
- Syntax: RENAME TABLE old_table_name TO new_table_name;

- Distinction: Separate from RENAME COLUMN used in ALTER.

3. DML (Data Manipulation Language) Commands

- **Purpose:** Insert, update, delete, and manage data within tables.

UPDATE

- Modifies existing rows.
- Syntax: UPDATE table_name SET column1 = value1 WHERE condition;
- **Critical:** WHERE clause prevents unintentional updates to all rows.
- Example (targeted): UPDATE employees SET salary = 70000 WHERE employee_id = 2;
- Example (mass update): UPDATE employees SET bonus = salary * 0.1;

DELETE

- Removes one or more rows.
- Syntax: DELETE FROM table_name WHERE condition;
- **Critical:** WHERE clause targets specific rows; omitting it deletes all rows.
- Example: DELETE FROM employees WHERE employee_id = 3;

4. Key Command Distinctions (DROP vs TRUNCATE vs DELETE)

Command	Purpose	Data Impact	Structure Impact
DROP	Permanently removes table	All data deleted	Table structure removed
TRUNCATE	Quickly deletes all rows	All data deleted	Table structure intact
DELETE	Removes specific rows (with WHERE)	Only targeted rows deleted	Table structure intact

5. MySQL Workbench: Safe Update Mode

- **Purpose:** Prevents accidental mass updates/deletions without a key column in WHERE clause.
- **Error:** You are using safe update mode...
- **Disabling Safe Update Mode:**
 1. Edit → Preferences → SQL Editor tab.
 2. Uncheck **Safe Updates**.
 3. Click OK and reconnect to DBMS.

- **Warning:** Disabling requires careful use of WHERE clauses.

6. DCL & TCL Overview

DCL (Data Control Language)

- **Purpose:** Manage user permissions (DBA task).
- **GRANT:** Gives privileges to users.
 - Syntax: GRANT privilege_name ON object_name TO user;
- **REVOKE:** Removes privileges.
 - Syntax: REVOKE privilege_name ON object_name FROM user;
- **Developer Note:** Attempting unauthorized actions triggers **access denied** errors.

TCL (Transaction Control Language)

- **Purpose:** Manage transactions to maintain data integrity.
- **COMMIT:** Saves all changes permanently.
- **ROLLBACK:** Reverts changes to last committed state or a savepoint.
- **SAVEPOINT:** Marks a point within a transaction to rollback partially if needed.

Session4. Sub Languages -3 & Integrity Constraints -1

1. Transaction Control Language (TCL)

Purpose: Manage transactions to ensure data integrity during operations like INSERT, UPDATE, DELETE.

1.1 Core TCL Commands

- **COMMIT:** Permanently saves all changes made in the current transaction.
- **ROLLBACK:** Reverts the database to the last COMMIT or a designated SAVEPOINT; acts as an "undo."
- **SAVEPOINT:** Creates a temporary marker within a transaction for partial rollbacks.

1.2 Role of AUTOCOMMIT

- **Autocommit Enabled:** Every operation (INSERT, UPDATE, DELETE) is immediately saved; ROLLBACK has no effect.
- **Disabling Autocommit:**
 - Command: SET AUTOCOMMIT = 0;
 - To re-enable: SET AUTOCOMMIT = 1;
 - Workflow: Disable autocommit → perform operations → use COMMIT to save or ROLLBACK to undo.

1.3 Practical Workflow Example

1. Table initially has 2 committed rows.
2. Disable autocommit: SET AUTOCOMMIT = 0;
3. Insert 2 uncommitted rows (rows 3 & 4).
4. Create savepoint: SAVEPOINT S1;

5. Insert 2 more rows (rows 5 & 6).
6. Partial rollback: ROLLBACK TO S1; → removes rows 5 & 6 only.
7. Table now has rows 1, 2, 3, 4; first two committed, next two pending.
8. Commit changes: COMMIT; and re-enable autocommit: SET AUTOCOMMIT = 1;

1.4 Real-World Use

- Developers use TCL for safety in uncertain/complex updates.
- SAVEPOINT is useful in multi-step transactions (e.g., banking operations).

2. Data Query Language (DQL)

- **Purpose:** Retrieve data from database tables.
- **Command:**
 - SELECT → displays/query data from one or more tables.

3. Integrity Constraints

Purpose: Enforce data accuracy, consistency, and reliability at the column level.

3.1 Core Constraints

1. **NOT NULL:** Column cannot have NULL values.
 - Syntax: CREATE TABLE table_name (column_name data_type NOT NULL, ...);
2. **UNIQUE:** Column must have distinct values.
 - Single column: column_name data_type UNIQUE

- Composite columns: UNIQUE (column1, column2)

3. **PRIMARY KEY:** Uniquely identifies each row; combines NOT NULL + UNIQUE.

- Only one primary key per table.
- Violations produce errors like: "duplicate entry '1' for key 'table_name.primary'".
- Additional unique columns can be defined as candidate keys.

4. **CHECK:** Validates data based on a condition.

5. **DEFAULT:** Provides a default value if none is specified.

6. **FOREIGN KEY:** Ensures referential integrity between tables.

3.2 Managing Primary Keys on Existing Tables

- **Add Primary Key:** ALTER TABLE table_name ADD PRIMARY KEY (column_name);
- **Remove Primary Key:** ALTER TABLE table_name DROP PRIMARY KEY;

3.3 Key Constraint Comparison

Constraint	Allows NULL	Allows Duplicates	Purpose
NOT NULL	No	Yes	Column must have a value
UNIQUE	Yes (usually one NULL allowed)	No	Ensures all values are distinct
PRIMARY KEY	No	No	Uniquely identifies each row

Session 5. Integrity Constraints -2

1. Overview of Integrity Constraints

- Integrity constraints enforce **data accuracy, consistency, and relational integrity** in SQL tables.
- Focused constraints in this session: **CHECK, DEFAULT, FOREIGN KEY**.
- These constraints are defined during table creation but can also be managed afterward.

2. CHECK Constraint

Purpose: Enforces column-level rules; only allows values that satisfy a specified condition.

Key Points:

- Validates data **at insertion or update time**.
- Different from a **WHERE clause**:
 - **CHECK:** Part of table structure; prevents invalid data.
 - **WHERE:** Filters existing data in queries.
- **Syntax:** column_name data_type CHECK (condition)
- **Example:** Ensure users are 18+

CREATE TABLE users (

user_id INT,

user_name VARCHAR(50),

age INT CHECK (age >= 18)

);

- Violation handling: Attempting to insert age < 18 triggers an error.

3. DEFAULT Constraint

Purpose: Automatically assigns a default value if none is provided during INSERT.

Key Points:

- Simplifies data entry; prevents NULLs when not desired.
- **Syntax:** column_name data_type DEFAULT default_value
- **Example:** Default order status to 'pending'

```
CREATE TABLE orders (
```

```
order_id INT,
```

```
order_status VARCHAR(20) DEFAULT 'pending'
```

```
);
```

- **Insertion Rules:**
 - Use column names when relying on default values:
 - INSERT INTO orders (order_id) VALUES (101);
 - Override default by providing a value explicitly:
 - INSERT INTO orders (order_id, order_status) VALUES (102, 'completed');

4. FOREIGN KEY Constraint

Purpose: Establishes relationships between tables; ensures referential integrity.

Key Concepts:

- Links a **child table column** to a **parent table primary key**.
- Prevents "orphan" records in child tables.
- **Parent Table:** Contains primary key (e.g., course_details with CID).
- **Child Table:** Contains foreign key referencing parent (e.g., students with course_id).

Syntax:

FOREIGN KEY (child_column_name) REFERENCES parent_table_name (parent_primary_key_column)

Example Workflow:

1. Create parent table:

```
CREATE TABLE course_details (  
  
    CID INT PRIMARY KEY,  
  
    course_name VARCHAR(50)  
  
);
```

2. Create child table with foreign key:

```
CREATE TABLE students (  
  
    student_id INT PRIMARY KEY,  
  
    student_name VARCHAR(50),  
  
    course_id INT,  
  
    FOREIGN KEY (course_id) REFERENCES course_details(CID)  
  
);
```

- Referential integrity enforced: inserting non-existent course_id fails.

5. Advanced Constraint Management

System-Generated Constraint Names

- SQL assigns automatic names (e.g., students_ibfk_1).
- View using:

SHOW CREATE TABLE table_name;

Dropping Tables with Foreign Keys

- Cannot drop a parent table if referenced by a child table.

Options:

1. Drop Child Table First:

DROP TABLE students;

DROP TABLE course_details;

2. Drop Foreign Key Constraint:

- Identify constraint: SHOW CREATE TABLE students;
- Remove constraint:

ALTER TABLE students DROP FOREIGN KEY students_ibfk_1;

- Then drop parent table:

DROP TABLE course_details;

☒ Summary

- **CHECK:** Enforces column-specific rules.
- **DEFAULT:** Auto-fills column values if missing.
- **FOREIGN KEY:** Maintains relationships and referential integrity.
- **Advanced management:** Use SHOW CREATE TABLE to identify constraints; handle parent-child dependencies carefully when dropping tables.

Session 6. Limitations of Excel& Intro to Real Time Database

1. Limitations of Excel for Data Management

- **Data Limit:** Maximum ~1 million rows per sheet; inadequate for large-scale operations.
- **Performance Issues:** Slows down significantly as data size increases; even basic operations like filtering become cumbersome.
- **Data Accessibility:** Retrieving specific data requires multiple filters; lacks the efficiency of SQL queries.
- **Data Redundancy:** Repetitive storage of the same information wastes memory.
 - Example: Supplier name repeated for every order in a sales sheet.
- **Data Consistency:** Excel does not enforce data types; allows invalid or irrelevant entries.
 - Example: Text like "high" in a numeric Price column.
- **Data Integrity:** No relational constraints; can create orphan or invalid records.
 - Example: An order referencing a non-existent customer ID is allowed.

2. Database Solutions: Normalization and Integrity

- **Normalization:** Breaks large tables into smaller, logical tables to reduce redundancy.
 - Example: Sales table split into:
 - Customer Details: customer_id, customer_name, address
 - Product Details: product_id, product_name, price
 - Supplier Details: supplier_id, supplier_name, manufacturer_name
 - Orders Table: order_id, order_date, customer_id, product_id, supplier_id

- Reduces memory usage and avoids repeating supplier or customer details.

- **Keys and Referential Integrity:**

- **Primary Key:** Unique identifier in a parent table.
- **Foreign Key:** Ensures child table references only valid parent records.
 - Example: Orders.customer_id must exist in Customer.customer_id.
- Prevents "orphan" records and maintains data accuracy.

3. Introduction to the "Maven Movies" Database

- **Purpose:** Sample database for practice; models DVD rental store or streaming platform film data.
- **Key Tables:**
 - actor → actor details (actor_id, first_name, last_name)
 - film → movie details (film_id, title, description, release_year, length, rating)
 - film_actor → relationship table linking actors to films
 - category & film_category → movie genres and relationships
 - Other tables: address, customer, rental, etc.

4. Foreign Key Implementation Methods

Method 1: Direct Foreign Key Inclusion

- Add foreign key column directly to a table.
- Example: students.course_id references course_details.course_id.

Method 2: Dedicated Relationship Table (Preferred)

- Creates a separate table to link two independent tables.
- Example from Maven Movies: film_actor table links actor and film.
 - Columns: actor_id (FK to actor), film_id (FK to film)
 - Avoids clutter and repetition in main tables.
- Benefits:
 - Cleaner, normalized structure.
 - Scales easily when a table needs multiple relationships (e.g., film → actor, category).

✓ Key Takeaways

- Excel is unsuitable for large-scale or complex data due to **size, performance, consistency, and integrity limitations**.
- Relational databases solve these issues with **normalization, primary/foreign keys, and structured relationships**.
- Using **relationship tables** is a scalable, real-world method to manage many-to-many relationships efficiently.

Session 7. Operators& Clauses -1

1. The Maven Movies Database Overview

- **Core Entity Tables:**

- actor: Actor details (actor_id, name).
- film: Movie details (film_id, title, rating, length/duration).
- customer: Customer info (customer_id, name, email, address_id).
- category: Movie genres (category_id, name).

- **Relationship (Junction) Tables:**

- film_actor: Links films and actors (film_id, actor_id).
- film_category: Links films and categories (film_id, category_id).

- **Transactional & Supporting Tables:**

- payment: Transactions (payment_id, customer_id, staff_id, amount, payment_date). Staff IDs 1 and 2.
- address, city, country: Address info linked via address_id.

2. The WHERE Clause: Foundation of Data Filtering

- Filters records row-by-row based on specified conditions.
- Essential for **SELECT**, **UPDATE**, and **DELETE** statements.
- Row-level evaluation:
 - Rows meeting the condition are included, updated, or deleted.
 - Rows failing the condition are ignored.

- **Basic Examples:**

- Fetch movies with rating 'G':
`SELECT * FROM film WHERE rating = 'G';`
- Fetch movies longer than 100 minutes:
`SELECT * FROM film WHERE length > 100;`

3. Conditional and Logical Operators

Operator	Alias	Function	Key Characteristic
AND	N/A	All conditions must be true	Short-circuits; stops checking if one condition is false
OR	N/A	At least one condition must be true	All conditions checked unless one is true
BETWEEN	Range	Filter within inclusive range	Inclusive of start & end values; numeric use
IN	Membership	Matches values in a list	Works for numbers and character strings
NOT	N/A	Negates a condition	Excludes data meeting a criterion

4. Operator Examples

• AND Operator:

- Fetch movies with rating 'G' **and** length > 100:
- `SELECT * FROM film WHERE rating = 'G' AND length > 100;`
- Both conditions must be true for a row to be returned.

• OR Operator:

- Fetch movies with rating 'G' **or** length > 100:
- `SELECT * FROM film WHERE rating = 'G' OR length > 100;`
- Returns a row if either condition is true.
- Correct usage for unique identifiers:
- `SELECT * FROM customer WHERE customer_id = 5 OR customer_id = 6;`

• BETWEEN Operator:

- Fetch payments for customer IDs 5 to 9:
- `SELECT * FROM payment WHERE customer_id BETWEEN 5 AND 9;`
- Inclusive of boundary values; concise alternative to multiple ORs.

• IN Operator:

- Fetch payments for specific non-sequential IDs:
- `SELECT * FROM payment WHERE customer_id IN (5, 6, 8, 9);`
- Fetch movies with rating in a set:
- `SELECT * FROM film WHERE rating IN ('G', 'PG', 'NC-17');`

- **NOT Operator:**

- Fetch all movies **except** rating 'G':
 - `SELECT * FROM film WHERE rating NOT IN ('G');`
-

5. Constructing Complex Queries

- Operators can be combined for multi-layered filtering.
 - Example: Fetch films that:
 1. Are not rated 'G', **AND**
 2. Have length between 50 and 100 minutes, **AND**
 3. Have rental duration in (5, 6, 7)
 - `SELECT * FROM film`
 - `WHERE rating NOT IN ('G')`
 - `AND length BETWEEN 50 AND 100`
 - `AND rental_duration IN (5, 6, 7);`
 - **Key Concept:** Using AND vs OR changes query logic drastically.
 - AND → All conditions must be met (more restrictive).
 - OR → Any condition met returns the row (less restrictive).
-

Session 8. Operators&Clauses -2

Core Principles of SQL Clauses

- Focuses on **ORDER BY, LIMIT, and DISTINCT** for sorting, restricting, and de-duplicating data.
- Highlights the **dual role of LIMIT**:
 - Restricting number of rows returned.
 - Skipping records to solve analytical problems (e.g., Nth highest/lowest).
- Stresses the **difference between SQL writing order vs. execution order** — a key interview concept.

ORDER BY Clause (Sorting Data)

- **Purpose:** Arrange results in ascending (ASC) or descending (DESC) order.
- **Default:** If order not specified → defaults to ascending.
- **Multi-level Sorting:**
 - First sort by one column.
 - For ties, sort by another column.
 - Example: ORDER BY customer_id ASC, amount DESC.
- **Interaction with WHERE:**
 - WHERE filters rows first.
 - ORDER BY sorts filtered results.
 - Example: staff_id = 1 payments sorted by amount.

LIMIT Clause (Restricting & Offsetting Results)

- **Use Case 1 – Restricting Rows:**
 - LIMIT n → returns top *n* rows.
 - Often used with ORDER BY for top/bottom N queries.
- **Use Case 2 – Skipping Rows (Offset):**

- LIMIT offset, count
- Skips *offset* rows, then returns *count* rows.
- Example: Skip first 4, return next 6 → rows 5–10.
- **Key Application – Nth Highest/Lowest:**
 - **Nth Lowest:** ORDER BY salary ASC → LIMIT n-1,1.
 - **Nth Highest:** ORDER BY salary DESC → LIMIT n-1,1.
 - Example: 3rd highest → skip top 2, fetch next 1.

DISTINCT Clause (Retrieving Unique Values)

- **Single Column:** Returns unique values only.
 - Example: Unique staff_id values.
- **Full Row:** Returns unique rows when applied with *.
 - Duplicate row = identical across all columns.
 - Example: Unique rows in employees table.

SQL Query Sequences

- **Writing Order (Syntactic)**
 1. SELECT
 2. DISTINCT
 3. FROM / JOIN
 4. WHERE
 5. ORDER BY
 6. LIMIT
- **Execution Order (Logical by Engine)**
 1. FROM / JOIN (get source tables)
 2. WHERE (filter rows)
 3. SELECT (pick columns)
 4. DISTINCT (remove duplicates)
 5. ORDER BY (sort results)

6. LIMIT (restrict/offset rows)

it is displayed.

Session 9. Installation of MySQL & Integrity Constraints -1

Executive Summary

- Session divided into **two parts**:
 1. **MySQL installation guidance**
 2. **Conceptual & practical lesson on SQL constraints**
- Installation: Students asked to follow a **YouTube tutorial** (“data analytics with DVM”) for reliable setup.
- Core technical focus: **6 SQL integrity constraints** → NOT NULL, UNIQUE, PRIMARY KEY, CHECK, DEFAULT, FOREIGN KEY.
- Purpose: Constraints ensure **data accuracy, prevent invalid entries, and enforce table relationships**.

1. MySQL Installation Guidance

- **Initial Approach**: Instructor tried **live installation via screen share**, but clarity issues made it ineffective.
- **Solution**: Students instructed to follow the **step-by-step installation video** on YouTube.
 - Channel: *data analytics with DVM*
 - Required: **MySQL Server + MySQL Workbench**
 - Confirmed compatibility with versions like **MySQL Workbench 8.0**.
- **Common Pitfall**:
 - A student mistakenly downloaded **Microsoft SQL Server** instead of MySQL.
 - Clarification: MySQL is an **Oracle product**, distinct from Microsoft SQL Server.
 - Fix: Uninstall wrong software → perform a **clean MySQL installation**.
- **Support**: Instructor to resolve any errors in future sessions.

2. SQL Integrity Constraints

Purpose of Constraints

- Enforce rules at the **database level**.
- Maintain **accuracy, consistency, and reliability** of data.
- Prevent improper/invalid entries.
- Define **logical relationships** between tables.

2.1 NOT NULL

- **Function:** Column must always have a value.
- **Syntax:** column_name datatype NOT NULL
- **Example:** customer_id INT NOT NULL → NULL insertion rejected.
- **Use Case:** Mandatory fields (e.g., employee IDs).

2.2 UNIQUE

- **Function:** Prevents duplicate values in a column (or set of columns).
- **Single-Column:** Add UNIQUE after datatype or at end of definition.
- **Multi-Column (Composite):** UNIQUE(col1, col2) ensures unique combinations.
- **Example:** manager_id set as UNIQUE → duplicate 2 rejected.

2.3 PRIMARY KEY

- **Function:** Combines NOT NULL + UNIQUE to uniquely identify records.
- **Properties:**
 - No NULL values.
 - No duplicate values.
 - Each table can have **only one PRIMARY KEY**.
- **Example:** category_id as PRIMARY KEY → NULL/duplicate insert fails.

2.4 CHECK

- **Function:** Restricts values to satisfy a condition.
- **Syntax:** CHECK (condition)
- **Example:** CHECK (age >= 18) → inserting age = 16 rejected.

2.5 DEFAULT

- **Function:** Assigns a default value if none is provided during insert.
- **Syntax:** DEFAULT value
- **Example:** order_status VARCHAR(20) DEFAULT 'pending'

- Insert without status → auto set to "pending".
- Can be overridden with explicit value.

2.6 FOREIGN KEY

- **Function:** Creates **relationships between tables**, enforcing referential integrity.
- **Concepts:**
 - **Parent Table** → contains PRIMARY KEY.
 - **Child Table** → contains FOREIGN KEY referencing parent.
 - Relationship: Foreign key value must exist in parent's primary key column.
- **Syntax:**
- FOREIGN KEY (child_column) REFERENCES parent_table(parent_column);
- **Example:** students.course_id as FOREIGN KEY referencing course_details.C_ID.
 - Ensures only existing course IDs can be inserted.

✓ This summary gives you **installation guidance, purpose of constraints, and each constraint's function, syntax, and example** in concise form.

Session 10. Integrity Constraints -2, Operators & Clauses -1

I. Maven Movies Practice Database

- **Purpose:** Real-time, sample DVD rental store database for exercises.
- **Theme:** Reflects DVD rental concepts; can be treated as OTT platform by ignoring rental-specific columns.
- **Key Tables:**
 - actor: actor_id, first_name, last_name.
 - film: film_id, title, description, release_year, length, rating.
 - payment: payment_id, customer_id, staff_id, amount.
 - film_actor (relationship table): Links actor_id and film_id.
- **Setup & Access:**
 1. Download the Maven Movies SQL file.
 2. Open in SQL Workbench (unconnected tab).
 3. Reconnect to DBMS.
 4. Select all script (Ctrl+A) and execute to create/populate tables.
 5. For future use, activate with USE Maven Movies.

II. Advanced Foreign Key Concepts

Parent Table Deletion Issue

- Cannot drop a parent table if its primary key is referenced by a child table's foreign key.
- Example: DROP TABLE course_details; fails if referenced by students.

Solutions

1. **Drop the Child Table First:** Remove the child table (e.g., students), then drop the parent.
2. **Remove Foreign Key Constraint:** Keep both tables but delete the foreign key relationship.

Removing a Foreign Key Constraint

- **Step 1:** Identify constraint name
 - Command: SHOW CREATE TABLE <table_name>;
- **Step 2:** Drop foreign key
 - Command: ALTER TABLE <table_name> DROP FOREIGN KEY <constraint_name>;

Relationship Architectures

1. **Direct Linking:** Add a foreign key column directly to the child table.

2. **Relationship Tables (Junction Tables):** Create a third table with only foreign keys from the two tables it connects.
- Example: film_actor links film and actor via film_id and actor_id.

III. Data Filtering with WHERE Clause

- **Purpose:** Filters rows in SELECT, DELETE, and UPDATE statements.
- **Behavior:** Evaluates conditions **row by row**; only rows meeting conditions are included.

Core Operators

Operator	Type	Purpose	Example	Notes
AND	Logical	Combines multiple conditions (all must be true)	WHERE rating='G' AND length>100	Both conditions required
OR	Logical	Combines multiple conditions (any true)	WHERE customer_id=5 OR customer_id=6	Retrieves multiple entities
BETWEEN	Range	Selects values within an inclusive range	WHERE customer_id BETWEEN 5 AND 9	Primarily numeric
IN	Membership	Matches against a list of values	WHERE rating IN ('G','PG','NC17')	Works for numeric/character, non-sequential values

IV. General SQL Syntax & Best Practices

- **Semicolon (;) Delimiter:**
 - Marks the end of a statement.
 - Mandatory for executing **multiple queries at once** to prevent errors.

V. Session Logistics & Next Steps

- **Next Session:** Operators and clauses, ORDER BY, LIKE operator.
- **Emphasis on Practice:** Write queries manually rather than copying to gain proficiency.
- **Resources Provided:** Shared folder with videos, scripts, and notes.

Session 11. Operators & Clauses -2

Executive Summary

- Focus on **SQL clauses**: ORDER BY, LIMIT, DISTINCT.
- Key takeaway: Difference between **query writing sequence** vs. **execution sequence** (important for optimization & interviews).
- Practical coverage: Troubleshooting common **MySQL beginner issues** (Safe Update Mode, correct use of UPDATE vs. INSERT).

1. SQL Clause Deep Dive

1.1 ORDER BY – Sorting Data

- **Function**: Sort records in ascending (ASC) or descending (DESC) order.
- **Default**: Ascending if not specified.
- **Multi-level sorting**: Sort by one column, then within equal values, sort by another.
 - Example:
 - `SELECT * FROM payment ORDER BY customer_id, amount DESC;`
- **Interaction with WHERE**: WHERE filters rows first → ORDER BY sorts filtered result.

1.2 LIMIT – Constraining Results

- **Use Case 1 – Restricting Rows (LIMIT n)**
 - Example:
 - `SELECT * FROM film LIMIT 20;`
- **Use Case 2 – Pagination / Skipping Records (LIMIT offset, count)**
 - Example:
 - `SELECT * FROM payment LIMIT 2, 4;`
 - `` → returns rows 3–6.
- **Application – Nth Highest Value**
 - Example: Second highest salary:
 - `SELECT * FROM employees ORDER BY salary DESC LIMIT 1, 1;`

1.3 DISTINCT – Retrieving Unique Values

- **Purpose:** Eliminates duplicate rows.
- **Syntax:** Used immediately after SELECT.
- **Examples:**
 - Single column:
 - `SELECT DISTINCT staff_id FROM payment;`
 - All columns:
 - `SELECT DISTINCT * FROM employees;`

2. SQL Query Structure and Execution Logic

2.1 Writing Sequence (Syntactic Order)

1. SELECT
2. DISTINCT
3. FROM / JOIN
4. WHERE
5. ORDER BY
6. LIMIT

2.2 Execution Sequence (Logical by Engine)

1. FROM / JOIN (source tables)
2. WHERE (filter rows)
3. SELECT (columns chosen)
4. DISTINCT (remove duplicates)
5. ORDER BY (sort results)
6. LIMIT (restrict final rows)

3. Practical Database Operations & Troubleshooting

3.1 Data Modification Techniques

- **UPDATE:** Modify existing rows (must use WHERE).

- **INSERT INTO table(column):** Adds new rows (must specify column names).
- **CASE Statement:** For multiple conditions (to be covered later).
- **Rule:** WHERE is valid only with SELECT, UPDATE, DELETE (not INSERT).

3.2 MySQL Safe Update Mode

- **Error:** Happens if UPDATE/DELETE is run without WHERE on a key column.
- **Purpose:** Prevents accidental modification of all rows.
- **Fix:** Disable in Workbench →
 1. Edit → Preferences → SQL Editor.
 2. Uncheck "Safe Updates".
 3. Reconnect to DBMS.

3.3 Database & Script Management

- **Best Practice:** Create dedicated databases for projects.
 - CREATE DATABASE db_name;
 - USE db_name;
 - CREATE TABLE ...
- **Saving Work:**
 - Queries run (CREATE/INSERT/UPDATE) → auto-saved to DB.
 - "Save" button in Workbench → saves .sql script locally.

✓ This summary covers **SQL clauses, query structure, troubleshooting, and best practices.**

Session 12. Like Operator & Functions -1

LIKE Operator & Pattern Matching

- **Purpose:** Performs pattern matching in WHERE clause searches.
- **Wildcards:**
 - % → zero, one, or many characters.
 - _ → exactly one character.
- **Common Use Cases:**
 - Names starting with "M":
 - Pattern: 'M%' → SELECT * FROM customer WHERE first_name LIKE 'M%';
 - Names ending with "M":
 - Pattern: '%M'
 - Names starting & ending with "M":
 - Pattern: 'M%M'
 - Fixed-length match with _:
 - 'T_M' → finds "Tom", "Tim".
 - 'M__Y' → finds "Mary".
 - Word length restriction:
 - 'A____' → 7-letter country starting with "A".
 - Combining wildcards:
 - 'A%I_' → country starting with "A" and second-last character "I".
- **Note:** Regular expressions (advanced pattern matching) mentioned as a future topic.

SQL Functions (Focus on Numeric & Aggregate)

- **Aggregate Functions:** Operate on sets of values → return single summary.
 - MIN() → smallest value.

- MAX() → largest value.
- COUNT() → number of rows (or values in a column).
- SUM() → total of numeric column.
- AVG() → average of numeric column.
- **Multiple aggregates** can be combined in one query.
 - Example: SELECT MIN(amount), MAX(amount), SUM(amount), AVG(amount) FROM payment;
- **Column Aliases:**
 - Syntax: AS keyword.
 - Example: SELECT MAX(amount) AS 'Maximum Amount' FROM payment;.

ROUND Function

- **Purpose:** Rounds decimals to nearest integer or specified decimal places.
- **Logic:**
 - .5 or higher → rounds up.
 - < .5 → rounds down.
- **Usage:**
 - On numbers: SELECT ROUND(5.8); → 6.
 - On columns: SELECT ROUND(amount) FROM payment;.
 - With precision: ROUND(value, decimals) → SELECT ROUND(10.3566, 2); → 10.36.

Ancillary SQL Commands

Data Manipulation Language (DML)

- **DELETE:** DELETE FROM table_name WHERE condition; → removes rows.
- **UPDATE:** UPDATE table_name SET column = value WHERE condition;
 - Complex updates: use CASE statement.
 - Issue: "safe update mode" blocks unsafe updates (without key/WHERE).

Data Definition Language (DDL)

- **ALTER TABLE** allows structural modifications:
 - MODIFY → change column data type.

- RENAME COLUMN → rename existing column.
- DROP COLUMN → remove a column.

Troubleshooting & Best Practices

- **Safe Update Mode:** Protection against accidental mass updates/deletes.
- **Debugging:** Save SQL + error message for easier fixes.
- **Practice Emphasis:** Regular practice is key for mastering SQL syntax.

Session 13. Functions -2

Numeric functions — Rounding & truncation

- **ROUND(value[, precision])**
 - Standard rounding: .5 or greater → round up; < .5 → round down.
 - Accepts optional second parameter to set decimal places (e.g., ROUND(10.3566,2) → 10.36).
- **CEIL(value)** (ceiling)
 - Always rounds **up** to the next integer (e.g., CEIL(6.2) → 7, CEIL(6.8) → 7).
 - **Cannot** accept a second parameter for decimal places.
- **FLOOR(value)**
 - Always rounds **down** to the integer part (e.g., FLOOR(6.2) → 6, FLOOR(6.8) → 6).
 - **Cannot** accept a second parameter.
- **Note:** Mnemonic — CEIL = up (ceiling), FLOOR = down (floor).

Numeric functions — Remainder

- **MOD(dividend, divisor)**
 - Returns remainder: MOD(4,2) → 0, MOD(3,2) → 1, MOD(3.2,2) → 1.2.
 - Advanced interview use-cases with MOD noted for future sessions.

Character (string) functions

- **Substring extraction**
 - LEFT(string, n) → first n chars (returns whole string if n > length).
 - RIGHT(string, n) → last n chars.
 - SUBSTR(string, start_pos, n) / MID() → extract n chars starting at start_pos (**1-indexed**, spaces count).
- **Case conversion & length**
 - LOWER() / LCASE() → lowercase; UPPER() / UCASE() → uppercase.
 - LENGTH(string) → number of characters.

- **Concatenation**

- CONCAT(a, b, ...) → combine fields/strings (can mix text, numbers, dates).
- Example with separator: CONCAT(first_name, ' ', last_name) AS full_name.
- Common real-world use: combine address parts into one field.

- **Trimming spaces**

- TRIM() → remove leading & trailing spaces.
- LTRIM() → remove leading spaces only.
- RTRIM() → remove trailing spaces only.

Date functions

- **Extracting components**

- YEAR(date) → year, MONTH(date) → numeric month (1–12), MONTHNAME(date) → full month name ('May').
- Example: Count rentals in May:
- SELECT COUNT(*) FROM rental WHERE MONTHNAME(rental_date) = 'May';
- -- OR
- SELECT COUNT(*) FROM rental WHERE MONTH(rental_date) = 5;

- **Current date/time**

- NOW() → current date **and** time.
- CURRENT_DATE → date only.
- CURRENT_TIME → time only.
- SYSDATE() → system date/time (may differ if system clock differs).

- **Date difference**

- DATEDIFF(date1, date2) → returns difference in **days**.
- **Order matters:** put the **latest** (most recent) date first to get a positive result (e.g., DATEDIFF(return_date, rental_date)).
- Practical uses: order fulfillment time, employee tenure (DATEDIFF(NOW(), date_of_joining)), rental durations.

Combined usage & business examples

- Functions are frequently **combined** for real tasks, e.g.:
 - Calculate average rental duration per month using AVG(DATEDIFF(...)) grouped by MONTHNAME(...).
 - Clean and concatenate address fields (TRIM + CONCAT) for reporting.
 - Round financial metrics for display (ROUND(SUM(amount), 2)).

Practical limitations & gotchas

- **CEIL/FLOOR** do **not** accept a precision parameter — only operate on integers.
- **SUBSTR / MID** start index is **1**, not 0.
- **CONCAT** auto-coerces non-strings but you may want explicit formatting for dates/numbers.

Future topics called out

- **ADDDATE()** — adding intervals to dates.
- Advanced interview uses of **MOD()**.

Session 14. Functions -3 & Regular Expression

Date functions — key points

- **DATEDIFF(latest_date, older_date)**
 - Returns the number of **days** between two dates.
 - **Order matters:** put the most recent date first to get a positive result.
 - Examples:
 - Rental duration: `DATEDIFF(return_date, rental_date)`
 - Employee experience: `DATEDIFF(NOW(), date_of_joining)`
- **Approximation method** (to convert days to larger units)
 - $\text{Years} \approx \text{DATEDIFF}(...) / 365$
 - $\text{Months} \approx \text{DATEDIFF}(...) / 30$
 - Explicitly approximate — leap years/month-length variations not accounted for.
- **ADDDATE(date, INTERVAL n UNIT)**
 - Adds an interval (days, months, etc.) to a date.
 - Examples: `ADDDATE(rental_date, INTERVAL 10 DAY)`, `ADDDATE(rental_date, INTERVAL 1 MONTH)`
 - Instructor note: used less frequently in real-time than `DATEDIFF`.

REGEXP vs LIKE — quick comparison

- LIKE uses % (any number) and _ (single char).
- REGEXP uses regular-expression syntax: .* (any number), . (single char), plus anchors/operators — more expressive.
- For the shown examples, performance differences were not significant; pick the tool based on complexity & personal comfort.

Core REGEXP anchors & operators (with examples)

- **^ — starts with**
 - Example: `WHERE title REGEXP '^S'` (titles beginning with S)
- **\$ — ends with**
 - Example: `WHERE title REGEXP 'M$'` (titles ending with M)

- `.` — any single character; `*` — any sequence of characters
 - Example: WHERE title REGEXP '^A.*H\$' (starts with A, ends with H)
- `[]` — character sets (like IN)
 - Example: WHERE title REGEXP '^[AEIOU]' (starts with a vowel)
 - Combine: WHERE title REGEXP '^[AEIOU].*[AEIOU]\$(starts & ends with a vowel)
- `|` — **OR** (either pattern)
 - Example: WHERE description REGEXP 'AT|ES' (contains "AT" or "ES")
 - Example: WHERE title REGEXP '^A|^E' (starts with A or starts with E)
- **Negation:** use NOT REGEXP to invert matches
 - Example: WHERE title NOT REGEXP '^[AEIOU]' (titles that do **not** start with a vowel)
- **Sequential matching:** placing patterns adjacent finds exact sequence positions
 - Example: WHERE title REGEXP '^S[AEIOU]' (starts with S and second char is a vowel)

REGEXP_REPLACE — targeted string edits

- **Syntax:** REGEXP_REPLACE(source_string, pattern, replacement)
- **Purpose:** Find a regex pattern in a string and replace it — ideal for small corrections inside large text fields.
- **Examples:**
 - Replace substring:
 - SELECT REGEXP_REPLACE('Capgemini', 'ni', 'ny');
 - -- returns: 'Capgeminy'
 - Replace whole word:
 - SELECT REGEXP_REPLACE('Capgemini Campus', 'Campus', 'Office');
 - -- returns: 'Capgemini Office'
- **Advantage vs UPDATE:** You can transform text inline (e.g., in a SELECT or an expression) or use it in an UPDATE for targeted replacements — often avoids rewriting entire large text blobs.

Practical notes & best practices

- REGEXP is far more expressive than LIKE (anchors, character classes, alternation, quantifiers) — learn it incrementally.

- When using DATEDIFF and converting to years/months, document that the result is **approximate** (leap days ignored).
- For small textual fixes across many rows, REGEXP_REPLACE can be safer and more efficient than full row rewrites.
- Always test regex patterns on sample data before applying updates to production tables.

Final note: "NotebookLM can be inaccurate; please double check its responses." — keep this verification mindset when applying regex/date logic in production.

Session 15. Joins -1

SQL Joins — Core Concepts

- **Purpose:** Combine related data from multiple tables into a single view.
- **Prerequisite:** Tables must share a **common column** to perform a join.
- **Real-world importance:** Central to RDBMS, ensures integrity & reduces redundancy.

Types of Joins

- **Inner Join** → Only matching records from both tables.
- **Outer Joins** (three subtypes):
 - **LEFT JOIN** → All records from left table + matching from right.
 - **RIGHT JOIN** → All records from right table + matching from left.
 - **FULL JOIN** → All records from both, match where possible.
- **Other joins:** NATURAL, CROSS (Cartesian), SELF — less common in practice.
- **Most used in practice:** INNER JOIN & LEFT JOIN.

Detailed Behavior

- **Inner Join** = intersection ($A \cap B$).
 - Example: Customer IDs {1,2,3,4} $\cap$ Age IDs {1,2,3,5} → {1,2,3}.
- **Left Join** = full left + matching right.
 - Non-matching right records → NULLs.
- **Right Join** = full right + matching left.
 - Non-matching left records → NULLs.
- **Full Join** = Left Join + Right Join.
 - Rarely used in practice.

Syntax & Implementation

- **General form:**
- `SELECT col_names`

- FROM table1 JOIN_TYPE table2
- ON table1.common_col = table2.common_col;
- **ON keyword** specifies the join condition.

Ambiguous Error

- **Cause:** Column name exists in both tables, not prefixed.
- **Error:** "column ID in the field list is ambiguous".
- **Resolution:**
 1. Use **full table name** (customer.id).
 2. Use **table aliases** (c.id), preferred for readability.

Full Join in MySQL

- **MySQL limitation:** No direct FULL JOIN keyword.
- **Workaround:** Combine **LEFT JOIN + RIGHT JOIN** with UNION.
- SELECT ...
- FROM customer c LEFT JOIN age a ON c.id = a.id
- UNION
- SELECT ...
- FROM customer c RIGHT JOIN age a ON c.id = a.id;

Practical Takeaways

- **INNER JOIN & LEFT JOIN** → most frequent in real-world usage.
- **RIGHT JOIN** → less common (can often be rewritten as LEFT).
- **FULL JOIN** → rare in practice.
- **Key skill:** Choosing the right join requires practice.
- **Next step:** Practical examples with Maven Movies DB to strengthen intuition.

Session 16. Joins -2 - Examples on Joins

Core Principles of SQL JOINS

- **Purpose:** Combine data from multiple tables into a unified dataset.
- **Join Type Selection:** Depends on the **analytical goal**:
 - For **matching (common) data** → INNER, LEFT, RIGHT often produce the same results.
 - For **non-matching (missing) data** → LEFT and RIGHT joins are essential (via NULL values).
- **Methodology for Join Queries (3-step approach):**
 1. Understand the data requirement.
 2. Identify common columns (PK/FK).
 3. Use a **relationship (junction) table** if direct columns are missing.

Challenge: No Direct Link Between Tables

- **Case Study:** Actor names with movies in Maven Movies DB.
 - actor table → actor_id.
 - film table → film_id.
 - No direct common column → **cannot join directly**.
- **Solution:** Use film_actor relationship table (contains actor_id, film_id) → enables **multi-table join**.

Systematic Query Building

- **3-step method applied:**
 - Requirement → actor + film names.
 - Common columns → via film_actor.
 - Multi-table join query:
 - SELECT f.title, a.first_name, a.last_name
 - FROM actor AS a
 - INNER JOIN film_actor AS fa ON a.actor_id = fa.actor_id
 - INNER JOIN film AS f ON fa.film_id = f.film_id;

Enhancing Queries with SQL Constructs

- **WHERE + LIKE** → filter by movie titles (e.g., WHERE title LIKE 'A%').
- **CONCAT()** → format full actor names (CONCAT(first_name, ' ', last_name) AS actor_name).
- **SUM() with WHERE** → aggregate customer payments (SUM(amount) WHERE first_name='Mary').
- **Integration principle:** Build queries by **layering multiple SQL functions** on top of joins.

Strategic Use of Join Types

- **When results are the same:**
 - INNER, LEFT, RIGHT produce identical results if focusing only on **existing (common) data**.
- **When LEFT/RIGHT are critical:**
 - To detect **missing/unavailable data** → leverage **NULLs**.
 - Examples:
 - **Unrecorded Ages:** LEFT JOIN customer → age, then WHERE age IS NULL.
 - **Non-Returned Orders:** LEFT JOIN orders → order_status, then WHERE returned IS NULL.
- **Key Syntax Note:** Use IS NULL, not = NULL.

Best Practices for Writing Queries

- **Use Aliases:** Shorten queries (actor AS a, film AS f) and resolve column name conflicts.
- **Iterative Development:** Start with SELECT * to confirm joins, then refine to required columns.
- **Code Formatting:** Space queries, align clauses for readability and error reduction.

✓ Key Takeaway:

- **INNER JOIN** → matching data.
- **LEFT/RIGHT JOIN** → powerful for analyzing **missing data via NULLs**.
- Building queries = joins + filters + formatting + aggregations.

Session 17. Joins -3 - Examples on Joins

● Core Theme

- JOINS are fundamental for solving **complex, multi-table data retrieval** problems.
- INNER JOIN → retrieves matching (available) data.
- LEFT/RIGHT JOIN → identifies non-matching (unavailable) data.
- Aliases are essential in outer joins to avoid incorrect results.
- Specialized joins (NATURAL, CROSS, SELF) have limited but specific relevance.
- Mastery of JOINS alone is sufficient to perform well in professional SQL roles.

1. Multi-Table Joins: Systematic Process

- **Logical Path Principle:** Build a chain of joins through common columns.
- **Case Example – Customers in India with names starting with A:**
 - No direct link between customer and country.
 - Path: Customer → Address → City → Country.
 - Each step leverages primary key/foreign key relationships.
- **Best Practice:** Choose the most efficient path (4 tables better than 5).
- **Scalability:** Same approach works for 2, 4, or even 6+ tables.
- **Query Example:**
 - INNER JOINS between customer, address, city, and country.
 - WHERE clause filters: first_name LIKE 'A%' AND country = 'India'.

2. Clarifying LEFT & RIGHT Joins (Unavailable Data)

- INNER JOIN → Only matching/common data.
- LEFT/RIGHT JOIN → Keep all rows from one table; NULLs appear for missing matches.
- **Use Case 1: Missing Ages**
 - LEFT JOIN customer to age.
 - WHERE age IS NULL → customers without age records.
- **Use Case 2: Orders Not Returned**

- LEFT JOIN orders to order_status.
- WHERE order_status.order_id IS NULL → correctly filters unmatched rows.
- **Insight:** Null checks must be done on the right-side table alias, not the base table.

3. Specialized Join Operations

- **Natural Join**
 - Pros: Very concise, hides duplicate common column.
 - Cons: Fails if multiple common columns exist; requires exact column name match.
 - Verdict: Unreliable → **not used in real-world industry**.
- **Cross Join**
 - Cartesian product (rows × rows).
 - Example: 4 × 4 tables = 16 rows.
 - Verdict: **No practical use** in analytics/business.
- **Self Join**
 - Table joined to itself using aliases.
 - Common in **hierarchical data** (e.g., Employee ↔ Manager).
 - Example: Employees E INNER JOIN Employees M ON E.Manager_ID = M.ID.
 - Useful in **HR analytics** but not everyday queries.

4. Strategic SQL Insights

- Multiple valid approaches (JOINS, subqueries, CTEs) → choose for clarity and performance.
- **Joins are the most straightforward and powerful** foundation for industry work.
- Proficiency comes only with **hands-on practice**.
- Strong JOIN knowledge alone allows survival in real-world SQL tasks.

5. Appendix: CHAR vs VARCHAR

- **CHAR(n):** Fixed-length, always reserves n bytes → wastes memory with shorter strings.
- **VARCHAR(n):** Variable-length, uses only required storage → memory efficient.
- Best Practice: Prefer VARCHAR unless fixed-length data is guaranteed.

Session 18. GROUP BY & Having Clause

Executive takeaways

- **Purpose:** GROUP BY summarizes rows into groups; HAVING filters those groups.
- **Key distinction:** WHERE filters rows **before** grouping; HAVING filters **after** aggregation.
- Correct clause order and understanding execution timing is essential for accurate analytical queries.

GROUP BY — core points

- Groups rows that share the same value(s) in one or more columns.
- Almost always used with aggregate functions: SUM(), COUNT(), MAX(), MIN(), AVG().
- **Best practice:** Include the grouped column(s) in SELECT so output is readable (e.g., SELECT staff_id, SUM(amount) ... GROUP BY staff_id).
- You can apply multiple aggregates in one query (e.g., SUM() and COUNT() together).
- Supports multi-column grouping for finer granularity (e.g., GROUP BY country, city).

HAVING — core points (vs. WHERE)

- **HAVING** filters **groups** produced by GROUP BY (can reference aggregates).
- **WHERE** filters **individual rows** prior to grouping and cannot use aggregate functions.
- Typical use: HAVING SUM(amount) > 150 to keep only groups whose aggregate meets a condition.

Query writing vs. execution order (important)

- **Writing (syntactic) order:** SELECT → FROM/JOIN → WHERE → GROUP BY → HAVING → ORDER BY → LIMIT.
- **Logical execution order:** FROM/JOIN → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT.
- Knowing both helps avoid errors and write efficient queries.

Practical examples (concise)

- **Total per staff:**
- SELECT staff_id, SUM(amount) AS total_collected

- FROM payment
- GROUP BY staff_id;
- **High-value customers (>150) and top 2:**
- SELECT first_name, last_name, SUM(amount) AS total
- FROM customer
- JOIN payment USING(customer_id)
- WHERE active = 1
- GROUP BY first_name, last_name
- HAVING SUM(amount) > 150
- ORDER BY total DESC
- LIMIT 2;
- **Aggregation after multi-table join (e.g., total by city in India):** join payment → customer → address → city → country, WHERE country = 'India', then GROUP BY city.

Tips & best practices

- Filter early with WHERE to reduce rows before aggregation (more efficient).
- Always test group logic on sample data (use SELECT * during development, then refine).
- Use clear aliases and explicit column names to avoid ambiguity in grouped queries.
- Document approximations (e.g., if converting day counts to years) when presenting aggregated metrics.

Learning resources

- Practice on challenge sites (HackerRank, LeetCode) to internalize grouping + having patterns.
- Build real examples (e.g., sales per region, top customers) to get intuition for grouping and filtering order.

Session 19. Sub Queries – 1

SQL Subqueries — Core Concepts

- **Definition:** A query nested inside another SQL query.
- **Execution flow:** Inner query runs first → its result is passed to the outer query.
- **Use cases:** Solving complex problems where results depend on intermediate queries.
- **Performance note:** Subqueries are often slower than JOINS.

1. Fundamentals

- **Two parts:**
 - **Inner Query (Subquery):** Executes first, returns result set.
 - **Outer Query:** Uses inner query's output for filtering/selection.
- **Syntax rules:**
 - Inner query must be enclosed in **parentheses ()**.
 - Enables **dynamic queries** (e.g., comparing salary against dynamically calculated average).
- **Best practice:** Break problem into steps → identify inner query → build outer query.

2. Types of Subqueries

- **Single-Row Subquery** → returns exactly one value.
- **Multi-Row Subquery** → returns multiple rows (often a single column).
- **Correlated Subquery** → depends on outer query values; very slow, discouraged in production.

3. Single-Row Subqueries

- **Key traits:**
 - Returns **one scalar value**.
 - Often uses **aggregate functions** (AVG, MAX, MIN, SUM, COUNT).
 - Works with comparison operators: =, >, <, >=, <=, !=.
- **Examples:**
 - Employees earning more than average salary.

- Second highest salary (via nested MAX).

4. Multi-Row Subqueries

- **Key traits:**
 - Returns a **set of values** (multiple rows, single column).
 - Uses **multi-row operators**, most commonly **IN**.
- **Examples:**
 - Employees whose salary matches any employee in department 2.
 - Employees working in departments with more than one employee.

5. Performance Considerations

- **Subqueries vs. JOINS:**
 - JOINS are usually **faster and more efficient**.
 - Subqueries may lead to slower execution times.
- **Correlated subqueries:**
 - **Very poor performance** → re-evaluated per row.
 - High **I/O overhead**.
 - Generally avoided in real-time applications.
- **Best practice:** Use JOINS or alternative methods when possible.

6. Practical Insights

- Subqueries are **powerful but tricky** to write.
- Every problem solvable by a subquery can also be solved with **joins/clauses**.
- Developer's choice depends on familiarity and readability.
- Conceptual understanding is essential, even if JOINS are preferred in practice.

Session 20. Sub Queries - 2 & Joins Assignment Solutions

1. Taxonomy of Subqueries

- **Non-Correlated Subqueries**

- Independent of the outer query.
- Can run as a standalone query.
- Types:
 - **Single-row subqueries** → return one value, often with aggregate functions, used with =, >, <.
 - **Multi-row subqueries** → return multiple values, require operators like IN, NOT IN.
- Efficient and widely used.

- **Correlated Subqueries**

- Dependent on outer query → cannot run independently.
- Inner query runs once per outer query row.
- Often use table aliases to pass outer query values to inner query.
- Execution behaves like a nested loop → highly inefficient.

2. Performance Impact of Correlated Subqueries

- **Execution Model**

- For each row in the outer query, the inner query re-runs.
- Example: Checking transactions for each customer across 599 rows → inner query repeated hundreds of times.

- **I/O Disconnects**

- Each input-output operation (like checking transactions for one customer) adds back-end workload.
- Multiplied across many rows → severe performance degradation.

- **Best Practices**

- Avoid correlated subqueries in production.
- Prefer **JOINS** instead.
- Developers should focus on single-row & multi-row subqueries and understand correlated ones conceptually (not practice them).

3. Principles of Efficient SQL Development

- **Memory management** → avoid queries that waste RAM.
- **Fast data retrieval** → prioritize performance in query design.
- **Input/Output efficiency** → minimize back-end processing load.

4. Practical Applications with JOINS (Maven Movies Examples)

- Count of unique film categories → COUNT(DISTINCT category_id).
- Actor names + films → multi-table INNER JOIN with CONCAT.
- Films + categories → INNER JOIN film, film_category, category.
- Number of films in "Action" → JOIN with WHERE name='Action'.
- Films + actors where length > 100 → JOIN + WHERE length > 100.
- Actors in "Animation" or "Children" → 4-table JOIN with WHERE name IN (...).

5. Ancillary Topics

- **Data constraints & types**
 - NOT NULL, UNIQUE, PRIMARY KEY apply to any datatype (numeric, char, date, etc.).
- **Role of Data Engineers**
 - Handle raw/unstructured data from clients (text, Excel).
 - Responsibilities:
 - Clean & preprocess data (EDA).
 - Create schemas, columns, and constraints.
 - Define surrogate keys (e.g., order_id, product_id).
 - SQL Developers/Analysts → work with structured databases (like Maven Movies).

✓ Key Takeaway:

- **Non-correlated subqueries** → useful & efficient.
- **Correlated subqueries** → conceptually important but **never recommended** in production.
- **JOINS** → the professional, scalable alternative.

Session 21. Views & ER - Diagram -1

1. SQL Views: Core Concepts

- **Definition:**
 - Stored SELECT query treated as a **virtual table**.
 - Does not store data itself → data remains in base tables.
 - Provides a reusable, named query structure.
- **Key Properties:**
 - Data integrity preserved → actual data stays in source tables.
 - Best practice: use descriptive naming (v_film_g, v_emp_highsalary).
- **Basic Operations:**
 - CREATE VIEW v_name AS SELECT ... → create a new view.
 - ALTER VIEW v_name AS SELECT ... → modify view definition.
 - DROP VIEW v_name → delete a view.
 - SELECT * FROM v_name → query a view like a normal table.

2. Use Cases for Views

- **Query Simplification & Reusability**
 - Encapsulates complex multi-table joins.
 - Speeds up query writing process.
 - Alternative: Stored Procedures can also simplify query logic.
- **Security & Access Control (Primary Motivation)**
 - Restricts user access to only specific columns/rows.
 - Prevents accidental or malicious UPDATE/DELETE on base tables.
 - Developers grant access **to views, not base tables**.

3. Ensuring Data Integrity: Non-Updatable Views

- **Risk of Standard Views**
 - Simple views on a single table are often **updatable**.
 - UPDATE or DELETE on a view modifies the **underlying table**.

- **Solution: Non-Updatable Views**

- Use DISTINCT or GROUP BY in view definition → makes it **read-only**.
- Example:
 - CREATE VIEW v_distinct_employee AS SELECT DISTINCT * FROM employees;
 - Attempting DELETE → fails with error ("not updatable").
- Guarantees secure, **read-only access** to sensitive data.

4. Entity-Relationship (ER) Diagrams

- **Definition:**

- Visual model representing a database schema.
- Shows **tables, columns, and relationships** between them.

- **Purpose:**

- Helps developers understand how tables connect.
- Displays metadata: constraints, indexes, and key relationships.

- **Generating ER Diagrams (MySQL Workbench):**

1. Go to **Database** → **Reverse Engineer**.
2. Select target database (e.g., Mavenmovies).
3. Run wizard → execute → finish.
4. View auto-generated ER diagram.

- **Interpreting Diagrams:**

- Tables shown as boxes with column lists.
- Lines = relationships (foreign key links).
- Symbols next to columns indicate PK, FK, or constraints.

Session 22. ER Diagram -2 & Indexes

1. Entity-Relationship (ER) Diagrams

- **Definition:** Graphical representation of a database schema showing tables, columns, and relationships.
- **Purpose:** Provides a high-level, visual understanding of database architecture.

1.1 Generation & Utility

- Created in MySQL Workbench using **Database → Reverse Engineer**.
- Utility:
 - Lists number of tables.
 - Shows columns in each table.
 - Maps relationships between tables.
 - Displays constraints like keys and nullability.

1.2 Symbols in ER Diagrams

- **Yellow Key** → Primary Key.
- **Red Key** → Column is both Primary Key & Foreign Key.
- **Blue Filled Diamond** → Not Null attribute.
- **Red Filled Diamond** → Not Null Foreign Key.
- **Blue Unfilled Diamond** → Nullable attribute.
- **Red Unfilled Diamond** → Nullable Foreign Key.
- Also shows **indexes** and metadata for deeper structural insight.

2. SQL Indexes for Performance Optimization

- **Definition:** Lookup structures that improve data retrieval speed.
- **Purpose:** Avoids full table scans → faster **SELECT** queries.

2.1 Clustered Index

- **Key Traits:**
 - Defines the physical storage order of table rows.
 - Only **one per table**.
 - Created **automatically** in MySQL when a **PRIMARY KEY** is defined.
- **Benefits:** Fast direct lookups on primary key values.

- **Behavior Example:**

- Table without PK → rows retrieved in insertion order.
- Table with PK → rows automatically sorted by key values.

2.2 Non-Clustered Index

- **Key Traits:**

- Created manually on non-primary key columns.
- Separate structure with sorted column values + pointers to data rows.
- Does **not** affect physical order of table rows.

- **How it Works:**

1. Search sorted index structure.
2. Retrieve pointer to actual row.
3. Fetch row directly from main table.

- **Disadvantage:** Extra memory/storage overhead → should only be used on frequently queried columns.

2.3 Index Management

- **Create Index:**

- CREATE INDEX index_name ON table_name(column_name);
- ALTER TABLE table_name ADD INDEX index_name (column_name);

- **Drop Index:**

- ALTER TABLE table_name DROP INDEX index_name;

- **System Behavior:**

- MySQL → PK creates clustered index automatically.
- Non-PK manual indexes = non-clustered indexes.

Session 23. Temporary Tables & CTE -1

1. Temporary Tables

- **Definition:** Session-scoped tables used for transient data storage.
- **Existence:** Last only for the session; automatically deleted when the session ends.
- **Functionality:** Support full SQL operations (SELECT, INSERT, UPDATE, DELETE).
- **Not in schema:** Not listed in permanent database schema.

1.1 Syntax

- **Creation:** CREATE TEMPORARY TABLE table_name (...);
- **Example:**
 - CREATE TEMPORARY TABLE sample (SID INT, sname VARCHAR(10));
 - INSERT INTO sample VALUES (1,'a'),(2,'b');
 - SELECT * FROM sample;

1.2 Primary Use Case – Removing Duplicate Records

- **Problem:** Simple DELETE removes all duplicates, including originals.
- **3-Step Solution:**
 1. **Isolate Unique Rows:**
 - CREATE TEMPORARY TABLE temp AS SELECT DISTINCT * FROM original;
 2. **Empty Original Table:**
 - TRUNCATE TABLE original;
 3. **Repopulate Original:**
 - INSERT INTO original SELECT * FROM temp;
- **Benefit:** Scalable method for de-duplicating large datasets.

1.3 Secondary Use Case – Trial & Error Testing

- Create a temporary table → copy relevant data → test updates/queries safely → apply to main table if successful.
- Avoids complex rollback mechanisms for testing.

2. Common Table Expressions (CTEs)

- **Definition:** Temporary, named result sets defined with WITH clause.
- **Scope:** Exists only for the duration of a **single SQL statement**.
- **Purpose:** Simplifies complex queries by breaking them into logical steps.
- **Non-persistent:** Not stored in schema.

2.1 Syntax

```
WITH cte_name AS (
```

```
    SELECT ...
```

```
)
```

```
SELECT * FROM cte_name;
```

2.2 Core Features

- Query-scoped (one execution only).
- Increases readability for multi-join or nested queries.
- Cannot be reused in separate queries like temporary tables.

3. Comparison: Temporary Tables vs CTEs

Feature	Temporary Table	CTE
Lifespan	Whole session	Single query execution
Creation Syntax	CREATE TEMPORARY TABLE ...	WITH cte_name AS (...)
Reusability	Can be reused across multiple queries	Only usable in the defining query
Purpose	Store & manipulate data across session	Organize & simplify complex queries

4. Ancillary SQL Techniques

- **Create Table from Query:** CREATE TABLE new_table AS SELECT ...;
- **Insert Data from Query:** INSERT INTO table SELECT ...; (used in duplicate removal step).

Session 24. CTE – 2

1. Introduction & Core Idea

- CTEs = **temporary, named result sets** valid only within a single query.
- Purpose: **break down complex SQL problems into logical, readable steps.**
- Analytical questions can be solved using:
 - Normal SQL methods (GROUP BY, HAVING, JOINS).
 - Subqueries (nested queries).
 - CTEs (WITH clause).
- **CTEs advantage:** readability and self-documentation.
- **Limitation:** rarely used in real-world projects; developers usually prefer standard joins and clauses.

2. Foundational Concepts: Temporary Data Structures in SQL

- **Temporary Tables**
 - Created for temporary storage.
 - Exist only for the session; removed when session ends.
 - Cannot be queried in future sessions.
- **CTEs**
 - Temporary result set with query-level scope.
 - Exists only during execution of one SQL statement.
 - Defined with WITH<cte_name> AS (<query_logic>).
 - Main query can reference the CTE like a normal table.

3. Case Study 1 – Employees Above IT Department Average Salary

- **Objective:** Find employees earning more than IT department's average salary.
- **Subquery Approach**
 - Outer query gets employees.
 - Inner query computes IT dept average salary.
 - WHERE compares salary with subquery result.
- **CTE Approach**

- CTE calculates IT dept average salary.
- Main query uses both **employee table** + CTE.
- Required because output needs columns from employee table + value from CTE.

4. Case Study 2 – Departments with Salary Total > 110,000

- **Objective:** List departments where total salary exceeds 110,000.
- **Normal Method (GROUP BY / HAVING)**
 - SUM() + GROUP BY department.
 - HAVING filters aggregated results.
- **Subquery (Derived Table)**
 - Subquery in FROM computes salary totals.
 - Outer query applies WHERE filter.
- **CTE Approach**
 - CTE defines dept salary totals (department_salaries).
 - Main query selects from CTE and filters with WHERE.
 - Only CTE is referenced, since it already has required fields.

5. CTEs vs. Subqueries – Comparison

- **Logic Placement**
 - Subquery: nested inside WHERE / FROM / SELECT.
 - CTE: defined upfront, referenced later.
- **Readability**
 - Subqueries: hard to read/debug when nested.
 - CTEs: improve readability with descriptive, logical blocks.
- **Performance**
 - Subqueries: may cause performance issues.
 - CTEs: generally no inherent disadvantage; may optimize better.

6. Real-World Application

- **Normal SQL methods** (joins, GROUP BY, HAVING) = most common.
- **CTEs = rare in practice** (“99% you can’t find CTEs in real-time”).
- Still useful conceptually for learning, flexibility, and reading others’ code.

7. Relevance for BI Tools

- BI tools (Power BI, Tableau): advanced SQL (CTEs, subqueries) not usually required.
- **Essential SQL skills for BI:**
 - Basic syntax + queries.
 - Strong knowledge of SQL functions.
 - Proficiency with JOINS for combining datasets.

Session 25. Stored Procedures & Window Functions -1

1. Stored Procedures – Automation & Reusability

- **Definition:** A group of SQL statements stored in the database, reusable on demand.
- **Purpose:** Automates repetitive/daily tasks, avoids rewriting queries.
- **Benefit over Views:**
 - Can accept **IN** (input) and **OUT** (output) parameters.
 - More dynamic and flexible than static Views.

1.1 Stored Procedures vs. Views

- **Views:**
 - Mainly for data security and simplified access.
 - Static, cannot accept parameters (e.g., fixed filter on movies starting with 'A').
- **Stored Procedures:**
 - Designed for task automation and reusable logic.
 - Dynamic – accept IN/OUT parameters to change behavior at runtime.
 - More powerful for flexible queries.

1.2 Syntax & Creation

- **DELIMITER:** Changes statement terminator temporarily (e.g., \$\$, //) so the procedure is parsed as one block.
- **CREATE PROCEDURE:** Defines procedure with a name and optional parameters.
- **BEGIN ... END:** Marks procedure body.
- **Semicolons:** Terminate individual queries inside the block.
- **Example:**
 - DELIMITER \$\$
 - CREATE PROCEDURE procedure_name(params)
 - BEGIN
 - -- SQL statements;
 - END \$\$
 - DELIMITER ;

1.3 Execution

- Executed with:
- CALL procedure_name(params);
- Modern database GUIs often provide parameter entry forms for ease of use.

1.4 Parameters

- **IN Parameter (Input)**
 - Declared as IN param_name datatype.
 - Passed at execution to make logic dynamic.
 - Example: WHERE rating = rate instead of hardcoding WHERE rating = 'G'.
- **OUT Parameter (Output)**
 - Declared as OUT param_name datatype.
 - Uses SELECT ... INTO to assign a result to the OUT variable.
 - Requires 2 steps:
 1. CALL procedure_name('input', @var);
 2. SELECT @var;
 - Less practical; usually easier to return result sets directly.

2. Window Functions – Introduction

- **Definition:** Perform calculations across sets of rows related to the current row.
- **Syntax:** Use OVER clause with ORDER BY.
- **Key Uses:** Ranking, sequential numbering, comparisons across rows.
- **Functions Introduced:**
 1. RANK()
 2. DENSE_RANK()
 3. ROW_NUMBER()
 4. LEAD()
 5. LAG()

2.1 Ranking Functions

- RANK()

- Assigns rank based on ordering.
- Handles ties by skipping ranks.
- Example: With tie at 5th place → sequence: 5, 5, 5, next rank = 8.
- **DENSE_RANK()**
 - Similar to RANK(), but does **not skip** ranks after ties.
 - Example: With tie at 5th place → sequence: 5, 5, 5, next rank = 6.
- **Key Distinction**
 - **RANK()** → skips ranks after ties.
 - **DENSE_RANK()** → continuous sequence, no skips.
 - Common interview question.

Function	Behavior with Ties	Example (tie at rank 2)
RANK()	Skips subsequent ranks	1, 2, 2, 4, 5
DENSE_RANK()	No skipping	1, 2, 2, 3, 4

SESSION 26. Window Functions -2

Foundational concepts & ranking

- **Window functions** compute values across a set of rows related to the current row; they require an OVER(...) clause.
- **OVER clause:** defines the window; may include PARTITION BY and ORDER BY.
- **RANK()**
 - Assigns ranks based on ordering.
 - Ties get the same rank **and** cause subsequent ranks to be **skipped**.
 - Example sequence with a tie: 1, 2, 2, 4, 5.
- **DENSE_RANK()**
 - Same tie behavior but **no ranks are skipped** after a tie.
 - Example: 1, 2, 2, 3, 4.
- **Interview point:** Distinguishing RANK() vs DENSE_RANK() is common.

ROW_NUMBER() — unique sequential index

- **Purpose:** Assigns a unique sequential integer to every row (1,2,3...), even for ties.
- **Use case:** Add an ID/index to raw data that lacks a primary key or to de-duplicate with ROW_NUMBER() filters.
- **Example:**
- ROW_NUMBER() OVER (PARTITION BY department ORDER BY salary DESC) AS rn

PARTITION BY — grouped/windowed analysis

- **Function:** Splits rows into partitions; window function restarts for each partition.
- **Common uses:** gender-wise ranking, department-wise salary ranking, per-customer sequences.
- **Syntax tip:** OVER (PARTITION BY <cols> ORDER BY <col>) — no comma between PARTITION BY and ORDER BY.
- **Benefit:** Enables subgroup-level calculations without collapsing rows (unlike GROUP BY).

Accessing adjacent rows: LAG() and LEAD()

- **LAG(column, offset, default)**
 - Fetches a value from *previous* row(s) within the same partition.
 - offset defaults to 1 (previous row); default returned when out-of-bounds (e.g., first row).
 - Example: LAG(salary, 1, NULL) OVER (PARTITION BY dept ORDER BY id)
- **LEAD(column, offset, default)**
 - Fetches a value from *next* row(s). Arguments are identical to LAG.
 - Example use: compare current value with next row's value for trend detection.
- **Edge behavior:** If no previous/next row exists, returned value is NULL (or the specified default).

Advanced application: LAG() + CASE for row-by-row comparisons

- **Pattern:** use LAG() to get prior-row value, then CASE to compare and produce a status column.
- **Example logic:**
 - LAG(salary) OVER (PARTITION BY dept ORDER BY id) AS prev_salary,
 - CASE
 - WHEN salary > prev_salary THEN 'Higher than previous'
 - WHEN salary < prev_salary THEN 'Lower than previous'
 - ELSE 'Same as previous'
 - END AS salary_status
- **Use case:** per-department salary movement, time-series difference flags, trend columns.
- **Benefit:** Enables row-level analytic flags without self-joins or procedural code.

Practical tips & placement

- **PARTITION BY** resets the window function per group — critical to get correct subgroup results.
- **Ordering matters** inside OVER(...); ranking & lag/lead depend on ORDER BY.
- **Combine functions** (e.g., ROW_NUMBER() with PARTITION BY) to implement de-duplication, top-N per group, etc.

Ancillary course & career notes

- **Course logistics:** notes emailed; videos uploaded to shared Drive (not emailed).
- **Hiring advice:** mastering SQL fundamentals up through JOINS covers ~90% of what early-career roles require.
- **Differentiators:** mentioning advanced topics (CTEs, window funcs) on a resume helps, even if deep practical grilling is unlikely.
- **Practice suggestions:** for 2–4 years experience roles, also practice LIKE and regex basics.
- **Debug tip:** create stored procedures first, then CALL them — don't run CREATE and CALL bundled in a single execution block.

✓ **Quick takeaway:** Window functions let you run powerful, row-aware analytics (rank, index, previous/next comparisons) without losing row-level detail — PARTITION BY + ORDER BY controls scope and sequence, and combining LAG() with CASE is a common, very practical pattern.

SESSION 27. Denormalization, Normalizations & Normal Forms

SQL, Data Analytics Careers, and Database Normalization — Key Insights

1. SQL Skill Development & Interview Strategy

- **Structured Learning Path:** Focus on fundamentals first, add advanced concepts gradually.
- **For Freshers**
 - **Essential:** Joins, operators, clauses, LIKE, regex, GROUP BY, HAVING.
 - **Secondary:** Views, indexes (basic understanding).
 - **Skip/De-emphasize:** CTEs, subqueries, temp tables (only high-level knowledge needed).
- **For Experienced (2–3+ years)**
 - Must solve problems with multiple approaches (e.g., LIKE vs regex).
 - Mention advanced topics (CTEs, subqueries) on resumes — adds credibility.
- **Interview Trends**
 - Focus: joins, clauses, operators, data patterns.
 - Advanced questions: mostly definitions, not full implementations.
 - Example answer: *“CTE is like a temporary table but exists only for one query execution.”*
- **Written Tests**
 - SQL questions common in MNCs/product companies.
 - Typically solvable with solid basics (functions, joins, operators).
 - Often sourced from **HackerRank** → consistent practice recommended.

2. Data Visualization & Analytics Career Path

- **Role Overview:** Data Analyst / Visualization Developer → create graphs, dashboards, reports.
- **Nature of Job:** SQL + visualization tools, minimal coding.
- **Core Skills**
 - SQL mastery.
 - One visualization tool (Tableau or Power BI).
 - Python optional (used in ~10–20% cases; not essential for most roles).
- **Career Advantages**
 - High demand, good salary packages.

- Smooth career switch from roles like QA.
- **Low stress:** fewer production issues, more work-life balance.
- Remote opportunities available via job portals.

3. Data Engineer vs Data Analyst

- **Data Workflow Stages**
 1. Data collection & preparation → **Data Engineer**.
 2. Processing & analysis → **Data Analyst**.
 3. Modeling & deployment → **Data Scientist**.
- **Data Engineer Responsibilities**
 - Collect raw data (Excel/text files).
 - Understand requirements.
 - Heavy cleaning: duplicates, nulls, outliers.
 - Apply **normalization** to structure data.
 - Build a “perfect database” for analysts.
- **Data Analyst Responsibilities**
 - Work on already cleaned data (may do ~20% extra cleanup).
 - Focus on analysis & dashboard creation.

4. Database Normalization — Technical Breakdown

- **Denormalization vs Normalization**
 - **Denormalization:** single big table, redundant, error-prone.
 - **Normalization:** break into smaller linked tables → reduced redundancy, improved integrity.
- **Data Anomalies in Denormalized Tables**
 - **Insertion anomaly:** can't add a product without an order.
 - **Updation anomaly:** price update must be repeated in thousands of rows.
 - **Deletion anomaly:** deleting an order may erase all info about a product.
- **Three Normal Forms (NFs)**
 1. 1NF (Atomicity & Uniqueness)

- No multi-valued cells, each cell holds a single value.
- Each row unique.

2. 2NF (No Partial Dependencies)

- Table already in 1NF.
- Non-key attributes must depend on the whole candidate key.
- Split off partial dependencies into separate tables.
- Example: order details vs product details.

3. 3NF (No Transitive Dependencies)

- Table already in 2NF.
- Non-key attributes shouldn't depend on other non-key attributes.
- Example: move delivery person details to a new table instead of linking via customer.

- **Result:** Raw, flat data → robust, relational, “perfect” database ready for analysis.

✅ Big Picture Takeaway:

- Freshers should focus on SQL fundamentals; advanced features matter more for resumes and senior roles.
- Data Analyst roles (SQL + Power BI/Tableau) offer high demand with low stress.
- Data Engineers handle the heavy lifting of cleaning & normalizing data; analysts leverage that clean data for insights.
- Database normalization (1NF → 3NF) is key to eliminating anomalies and ensuring efficiency.

SESSION 28. Interview Questions

18 Frequently Asked SQL Interview Questions — Key Insights

1. Data Types & Commands

- **CHAR vs VARCHAR**

- CHAR: fixed-length, occupies full defined memory.
- VARCHAR: variable-length, uses only required memory + small overhead.
- Key distinction: memory efficiency.

- **Data Removal Commands**

- DROP: removes table structure + data permanently.
- DELETE: removes specific rows via WHERE clause.
- TRUNCATE: removes all data but retains table structure.

- **Adding Columns:**

- ALTER TABLE table_name ADD COLUMN column_namedata_type;

2. Data Retrieval & Filtering

- **Range & Membership Operators**

- BETWEEN: selects values within a range.
- IN: matches values against a list.

- **Fetching Top N Records**

- ORDER BY + LIMIT (e.g., top 50 sales: ORDER BY sales DESC LIMIT 50).

- **Fetching Specific Rows (Range/Slice)**

- BETWEEN: SELECT * FROM film WHERE film_id BETWEEN 5 AND 8;
- LIMIT with offset: SELECT * FROM film LIMIT 4,4; (skip 4, show next 4).

- **Alternate Rows (Odd/Even)**

- MOD() function:
 - Odd: MOD(film_id, 2) = 1

- Even: $\text{MOD}(\text{film_id}, 2) = 0$

3. Pattern Matching & Joins

- **LIKE vs REGEXP**

- LIKE: simple pattern matching (WHERE last_name LIKE '%RA%').
- REGEXP: regex-based matching (WHERE last_name REGEXP 'RA').
- Example: 7-character countries → WHERE country LIKE '_____'.

- **Joins**

- Ambiguous column error occurs when identical column names exist in joined tables.
- Solution: prefix columns with table names or aliases.

4. Advanced SQL Concepts

- **WHERE vs HAVING**

- WHERE: row-level filtering (before GROUP BY).
- HAVING: group-level filtering (after GROUP BY).

- **RANK() vs DENSE_RANK()**

- RANK(): skips ranks after ties (e.g., 1,2,2,5).
- DENSE_RANK(): no skipped ranks (e.g., 1,2,2,3).

- **UNION vs UNION ALL**

- UNION: removes duplicates.
- UNION ALL: includes duplicates, faster.
- Columns in SELECT statements must match.

- **Correlated vs Non-Correlated Subqueries**

- Non-correlated: inner query independent; runs once.
- Correlated: inner query depends on outer; evaluated per row.

- **Temporary Tables vs CTEs**

- Temporary Table: exists for session duration.

- CTE: exists only for a single query.

5. Performance & Database Internals

- **Query Optimization**

- Goal: execute queries efficiently, retrieve data faster.
- Main technique: **indexes**.

- **Indexes**

- Clustered Index: automatically on primary key, data stored sorted.
- Non-Clustered Index: separate structure, pointers to data rows.
- View indexes: SHOW INDEX FROM table_name;

6. Miscellaneous Functions

- NOW(): returns current system date and time.

7. Career & Learning Takeaways

- Prioritize **fundamentals** (joins, operators, LIKE, regex) for freshers/1–2 years experience.
- Advanced topics (CTEs, subqueries) are important for senior roles.
- Hands-on practice is crucial (MySQL, sample databases like Maven Movies, IG Clone).
- Use public resources for practice: **HackerRank**, **Kaggle**, and online datasets.

Session 29. MCQ – 2

SQL 15 MCQs — Key Concepts & Takeaways

1. Data Retrieval & Performance

- **Indexes**
 - Speed up SELECT queries; like a book's index for fast lookup.
 - Trade-off: slows down INSERT/UPDATE as index needs updating.
 - Applicable to any column type (numeric or character).
 - **Clustered Index**: determines physical table order; auto-created on primary key in MySQL.
 - **Non-Clustered Index**: separate structure; does not affect physical order.
- **LIMIT & OFFSET** (Pagination)
 - LIMIT 10: returns first 10 rows.
 - LIMIT 24, 100: skip 24 rows, return next 100.
 - LIMIT 10 OFFSET 5: skip 5 rows, return next 10.
- **Clause Order**
 - Correct sequence: WHERE → GROUP BY → HAVING
 - WHERE: row-level filtering
 - GROUP BY: aggregate rows
 - HAVING: filters aggregated groups

2. Pattern Matching & Data Handling

- **LIKE Operator**
 - `_`: matches single character
 - `%`: matches zero or more characters
 - Examples:
 - Second character 'a': `_a%`
 - Exactly five characters: `_____`
 - Three-letter names with 'a' in the middle: `_a_`
- **Regular Expressions**
 - Example: match digits only → `^[0-9]*$`

- ^: start, [0-9]: digit, *: zero or more, \$: end

- **NULL Handling**

- Cannot use = or !=; must use IS NULL / IS NOT NULL
- Arithmetic with NULL → result is NULL
- NULL is not zero or FALSE

3. Database Objects

- **Table Properties**

- Max table name length: 64 bytes (≈ 64 characters)

- **Views**

- Virtual table; stores query, not data
- Queries base table on access

- **Temporary Tables**

- Scope: current session only
- Auto-deleted on session close
- Common use: removing duplicates

4. Advanced SQL Concepts

- **Subqueries**

- **Non-correlated:** inner query independent, runs once
- **Correlated:** inner query depends on outer query, runs per row

- **Window Functions**

- Operate across related rows (OVER clause)
- Key functions: RANK(), DENSE_RANK(), ROW_NUMBER(), LEAD(), LAG()
- **RANK() vs DENSE_RANK():**
 - RANK(): skips ranks after ties (e.g., tie for 2 → next rank 4)
 - DENSE_RANK(): no skipped ranks (e.g., tie for 2 → next rank 3)

5. MySQL-Specific Functions

- **Date & Time**

- NOW(): current date + time
- CURDATE(): current date
- CURRENT_TIME(): current time
- SYSDATE(): system date + time

SESSION 30. MCQ-1

MySQL & SQL — Key Principles & Takeaways

I. Core Data Retrieval and Filtering

- **SELECT Statement**
 - Fundamental command to extract/display data.
 - Colloquial terms like "get" or "extract" are incorrect in SQL.
- **WHERE Clause**
 - Filters rows based on conditions.
 - **AND**: All conditions must be true.
 - Example: find a user with specific first & last names.
 - **OR vs IN**:
 - Multiple ORs: WHERE team_one = 2 OR team_one = 4 ...
 - IN Operator (preferred): WHERE team_one IN (2,4,6,8)
- **Pattern Matching with LIKE**
 - %: zero or more characters, _: single character
 - Examples:
 - Names starting with 'S': LIKE 'S%'
 - Second letter 'r': LIKE '_r%'
- **NULL Handling**
 - Use IS NULL / IS NOT NULL, not = or !=.
 - Example: WHERE user_phone_number IS NULL
- **ORDER BY Clause**
 - Sort results ascending (ASC) or descending (DESC).
 - Syntax: SELECT ... FROM ... WHERE ... ORDER BY column_name DESC

II. Data Aggregation and Grouping

- **Aggregate Functions**
 - MAX(): maximum value
 - MIN(): minimum value

- AVG(): average value
- SUM(): total sum
- COUNT(): number of rows matching criteria
- **GROUP BY Clause**
 - Groups rows sharing column values for aggregation
 - Example: group sales by region
- **HAVING Clause**
 - Filters aggregated results (after GROUP BY)
 - Distinct from WHERE, which filters individual rows before aggregation

III. Data Integrity and Table Structure

- **Integrity Constraints**
 - PRIMARY KEY: unique, non-NULL identifier
 - NOT NULL: column cannot contain NULL
 - UNIQUE: all values must be distinct
 - FOREIGN KEY: enforce relationships between tables
 - CHECK: ensure column values meet a condition
 - DEFAULT: set default value for column
- **Modifying Table Structure**
 - ALTER TABLE command can add constraints or modify table structure
 - Example: adding a constraint to an existing table

IV. Transaction Control and Database State Management

- **TCL Commands**
 - COMMIT: permanently save all transactions
 - ROLLBACK: revert to last committed state or a savepoint
 - SAVEPOINT: set a temporary point to rollback partially

V. Advanced SQL Concepts

- **JOIN Operations**
 - Combine rows from multiple tables based on a common column
 - INNER JOIN: returns records with matching values in both tables
 - Example: join employee table (E) with company table (C) on ID
- **Stored Procedures**
 - Save & reuse SQL statements for modular execution
 - CALL command executes procedures
 - Can accept IN (input) and OUT (output) parameters
 - Example: CALL test();
- **String Manipulation Functions**
 - TRIM(): remove spaces from both ends
 - LTRIM(): remove spaces from left side
 - RTRIM(): remove spaces from right side
- **Auxiliary Concepts**
 - **Views:** virtual tables based on a stored query, simplify queries, enhance security
 - **Indexes:** speed up SELECT queries, may slow down INSERT/UPDATE/DELETE